Universidad Católica del Uruguay
Facultad de Ingeniería y Tecnologías

Materia: Sistemas Operativos
Alumno: Facundo Rodríguez
Laboratorio: Laboratorio 3 – Docker
Fecha: 26/03/2026

Índice

1. Introducción **1**
2. Desarrollo del Laboratorio **1**
2.1 Instalación de Docker 1
2.2 Verificación de funcionamiento 3
2.3 Ejecución de hello-world 4
2.4 Contenedor nginx 5
2.5 Acceso al contenedor 6
2.6 Gestión de contenedores e imágenes 7
2.7 Creación de imagen personalizada 8
2.8 Contenedor Ubuntu 11
2.9 Contenedor Alpine 12
2.10 Publicación en GitHub 13
3. Introducción a Docker **14**
4. Comandos fundamentales de Docker **15**
5. Dockerfiles: propósito y estructura **15**
6. Instrucciones más comunes de un Dockerfile **15**
7. Aplicación práctica de los comandos **16**
8. Conclusión **16**
9. Anexos **17**

1. Introducción

El presente laboratorio tiene como objetivo introducir el uso de Docker como herramienta de virtualización basada en contenedores. A lo largo del mismo se trabajó con la instalación de Docker Desktop, la ejecución de contenedores predefinidos, el uso de comandos básicos, la creación de imágenes personalizadas mediante Dockerfile y la ejecución de aplicaciones en entornos aislados.

Docker permite encapsular aplicaciones junto con sus dependencias, facilitando su portabilidad y ejecución en distintos entornos. En este laboratorio se busca comprender el funcionamiento de imágenes y contenedores, así como su gestión y uso práctico.

2. Desarrollo del Laboratorio

2.1 Instalación de Docker

Se ejecuta el instalador de Docker Desktop para comenzar el proceso de instalación.

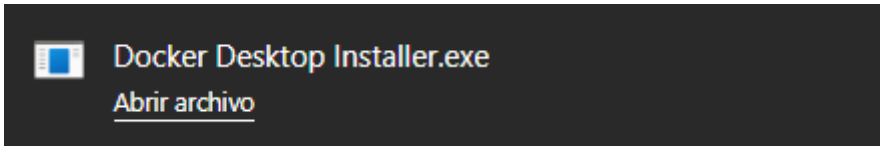

Figura 1. Captura instalador

Durante la instalación, Docker descomprime y configura los archivos necesarios para su funcionamiento.

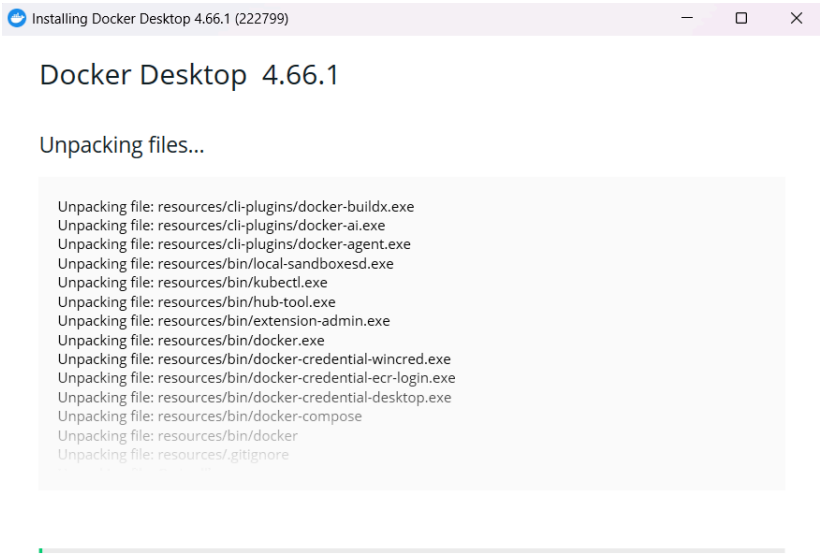

Figura 2. Unpacking

La instalación finaliza correctamente, requiriendo reiniciar el sistema.

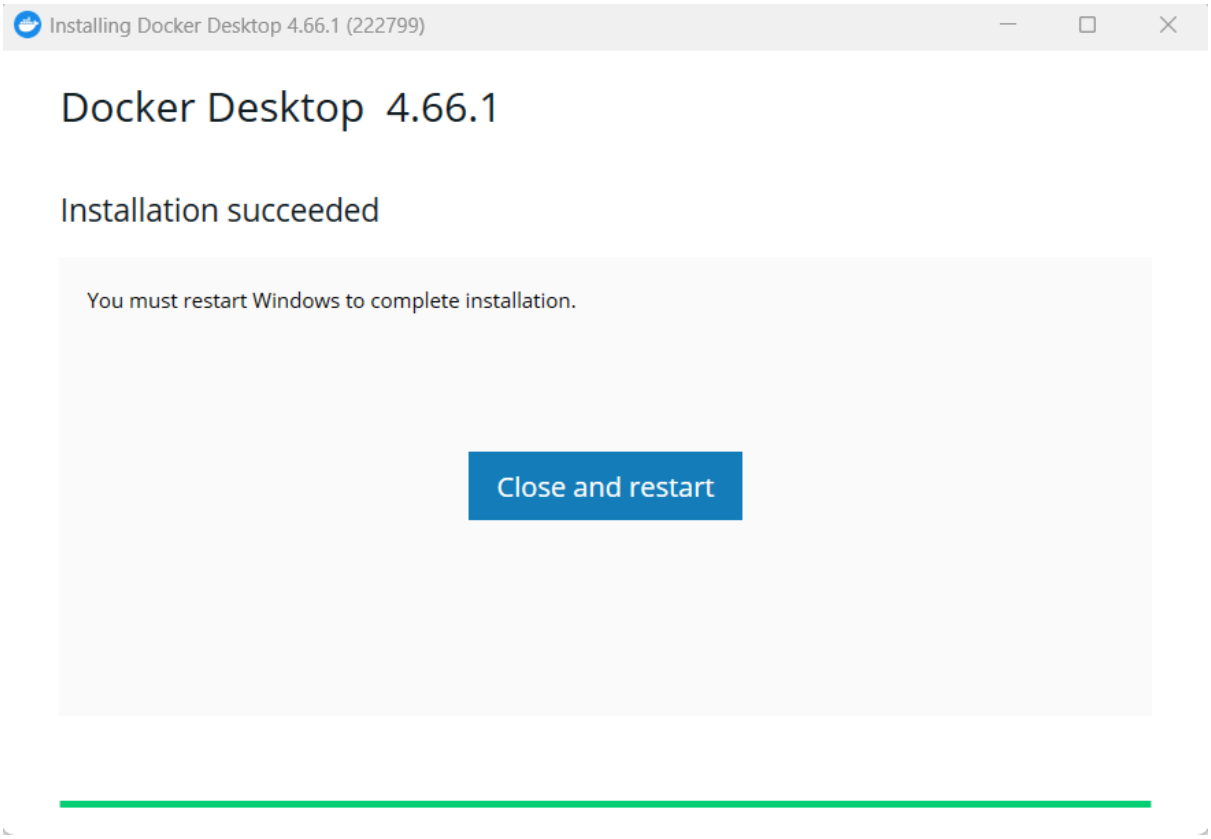

Figura 3. Installation succeeded

Se verifica la instalación mediante el acceso directo generado.

Figura 4. Icono Docker

Se inicia Docker Desktop verificando que el motor se encuentra activo.

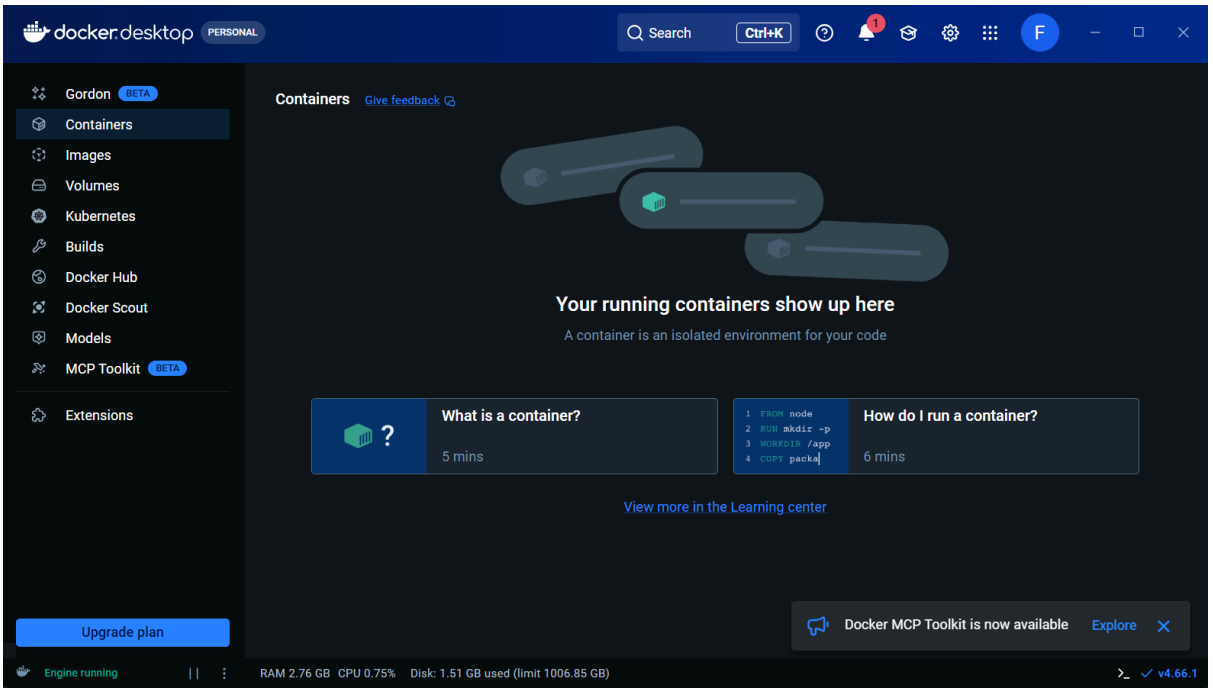

Figura 5. Interfaz Docker

2.2 Verificación de funcionamiento

Se verifica la instalación mediante la versión de Docker instalada.

```
C:\Users\facun>docker --version
Docker version 29.3.1, build c2be9cc
```

Figura 6. docker –version

Se obtiene información detallada del entorno Docker.

```
C:\Users\facun>docker info
Client:
 Version:    29.3.1
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  agent: Docker AI Agent Runner (Docker Inc.)
    Version:  v1.32.4
    Path:     C:\Program Files\Docker\cli-plugins\docker-agent.exe
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.20.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.32.1-desktop.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-buildx.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v5.1.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-compose.exe
  debug: Get a shell into any image or container (Docker Inc.)
    Version:  0.0.47
    Path:     C:\Program Files\Docker\cli-plugins\docker-debug.exe
  desktop: Docker Desktop commands (Docker Inc.)
    Version:  v0.3.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-desktop.exe
```

Figura 7. docker info

2.3 Ejecución de hello-world

Se ejecuta el contenedor de prueba, verificando el correcto funcionamiento del sistema.

```
C:\Users\facun\Lab03-Docker-Facundo>docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:452a468a4bf985040037cb6d5392410206e47db9bf5b7278d281f94d1c2d0931
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Figura 8. Ejecución de contenedor hello-world

No se observan contenedores en ejecución.

```
C:\Users\facun\Lab03-Docker-Facundo>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

Figura 9. docker ps

Se observa el contenedor hello-world en estado detenido.

```
C:\Users\facun\Lab03-Docker-Facundo>docker ps -a
CONTAINER ID   IMAGE         COMMAND    CREATED              STATUS                          PORTS     NAMES
c4c1fcb451d5   hello-world   "/hello"   About a minute ago   Exited (0) About a minute ago             trusting_wescoff
```

Figura 10. docker ps -a

2.4 Contenedor nginx

Se descarga y ejecuta la imagen nginx desde Docker Hub.

```
C:\Users\facun\Lab03-Docker-Facundo>docker run -d --name mi-nginx -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
5e815e07e569: Pull complete
bb3d0aa29654: Pull complete
587e3d84dbb5: Pull complete
510ddf6557d6: Pull complete
cde7a05ae428: Pull complete
3189680c601f: Pull complete
ec781dee3f47: Pull complete
669e0ab8e7fa: Download complete
96a6cfe061e0: Download complete
Digest: sha256:7150b3a39203cb5bee612ff4a9d18774f8c7caf6399d6e8985e97e28eb751c18
Status: Downloaded newer image for nginx:latest
023630cc777036cb84623c10b9b55aff35b91f3218949e6ea7873fc7bb804c47
```

Figura 11. Ejecución de nginx

Se verifica que nginx se encuentra activo.

```
C:\Users\facun\Lab03-Docker-Facundo>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                       NAMES
023630cc7770   nginx     "/docker-entrypoint.…"   48 seconds ago   Up 46 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp     mi-nginx
```

Figura 12. Contenedor en ejecución

Se observan contenedores activos y detenidos.

```
C:\Users\facun\Lab03-Docker-Facundo>docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS                      PORTS                                       NAMES
023630cc7770   nginx         "/docker-entrypoint.…"   59 seconds ago   Up 57 seconds               0.0.0.0:8080->80/tcp, [::]:8080->80/tcp     mi-nginx
c4c1fcb451d5   hello-world   "/hello"                 3 minutes ago    Exited (0) 3 minutes ago                                                trusting_wescoff
```

Figura 13. Listado completo de contenedores

Se accede a nginx mediante localhost, confirmando su funcionamiento.

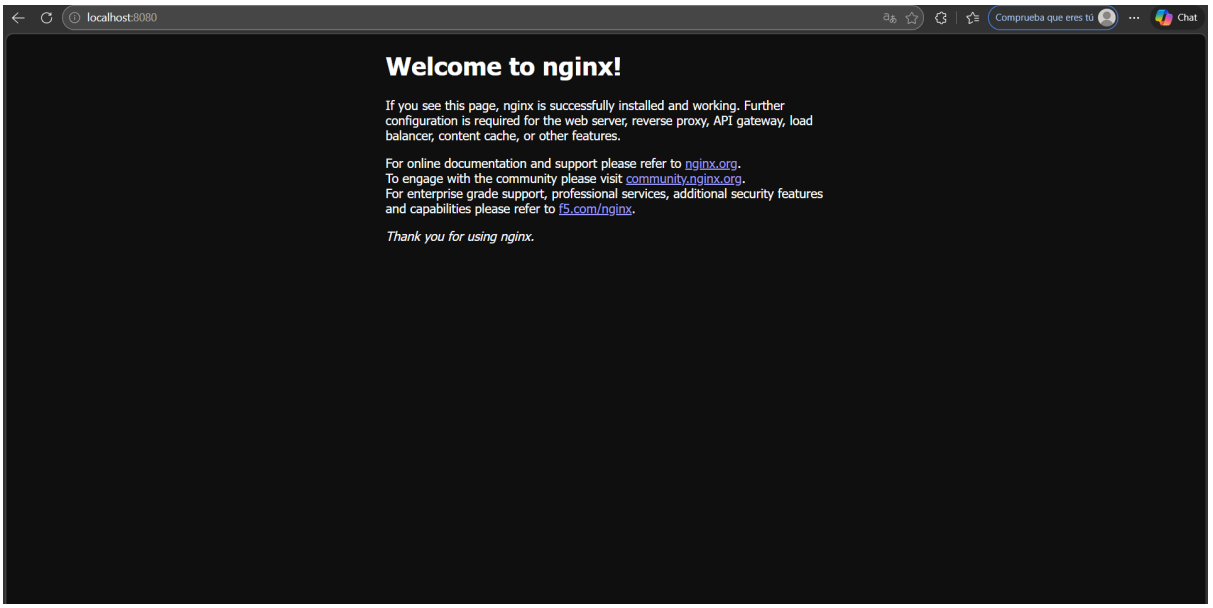

Figura 14. Acceso desde navegador

Se visualizan los registros del contenedor nginx.

```
C:\Users\facun\Lab03-Docker-Facundo>docker logs mi-nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/03/26 16:10:59 [notice] 1#1: using the "epoll" event method
2026/03/26 16:10:59 [notice] 1#1: nginx/1.29.7
2026/03/26 16:10:59 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/03/26 16:10:59 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/03/26 16:10:59 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/03/26 16:10:59 [notice] 1#1: start worker processes
2026/03/26 16:10:59 [notice] 1#1: start worker process 29
2026/03/26 16:10:59 [notice] 1#1: start worker process 30
2026/03/26 16:10:59 [notice] 1#1: start worker process 31
2026/03/26 16:10:59 [notice] 1#1: start worker process 32
2026/03/26 16:10:59 [notice] 1#1: start worker process 33
2026/03/26 16:10:59 [notice] 1#1: start worker process 34
2026/03/26 16:10:59 [notice] 1#1: start worker process 35
2026/03/26 16:10:59 [notice] 1#1: start worker process 36
172.17.0.1 - - [26/Mar/2026:16:12:25 +0000] "GET / HTTP/1.1" 200 896 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) C
hrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0" "-"
172.17.0.1 - - [26/Mar/2026:16:12:26 +0000] "GET /favicon.ico HTTP/1.1" 404 555 "http://localhost:8080/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWeb
Kit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0" "-"
2026/03/26 16:12:26 [error] 29#29: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 172.17.0.1, server: localhos
t, request: "GET /favicon.ico HTTP/1.1", host: "localhost:8080", referrer: "http://localhost:8080/"
```

Figura 15. Logs del contenedor

2.5 Acceso al contenedor

Se accede al contenedor de forma interactiva.

```
C:\Users\facun\Lab03-Docker-Facundo>docker exec -it mi-nginx bash
```

Figura 16. Acceso con docker exec

Se observa la estructura de archivos del sistema Linux.

```
root@023630cc7770:/# ls
bin  boot  dev  docker-entrypoint.d  docker-entrypoint.sh  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
```

Figura 17. Estructura del sistema (ls)

Se verifica que el directorio actual es la raíz (/).

```
root@023630cc7770:/# pwd
/
```

Figura 18. Directorio actual (pwd)

Se finaliza la sesión interactiva.

```
root@023630cc7770:/# exit
exit
```

Figura 19. Salida del contenedor

2.6 Gestión de contenedores e imágenes

Se detiene el contenedor nginx.

```
C:\Users\facun\Lab03-Docker-Facundo>docker stop mi-nginx
mi-nginx
```

Figura 20. Detención del contenedor

Se vuelve a iniciar el contenedor detenido.

```
C:\Users\facun\Lab03-Docker-Facundo>docker start mi-nginx
mi-nginx
```

Figura 21. Reinicio del contenedor

Se muestran las imágenes disponibles en el sistema.

```
C:\Users\facun\Lab03-Docker-Facundo>docker images
                                                                                    i Info →   U  In Use
IMAGE               ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest  452a468a4bf9       25.9kB         9.49kB    U
nginx:latest        7150b3a39203        240MB         65.8MB    U
```

Figura 22. Listado de imágenes

No se puede eliminar una imagen en uso por un contenedor.

```
C:\Users\facun\Lab03-Docker-Facundo>docker rmi hello-world
Error response from daemon: conflict: unable to delete hello-world:latest (must be forced) - container c4c1fcb451d5 is using its referenced image 452a468a4b
f9
```

Figura 23. Error al eliminar la imágen

Se elimina el contenedor asociado y luego se elimina correctamente la imágen.

```
C:\Users\facun\Lab03-Docker-Facundo>docker rm c4c1fcb451d5
c4c1fcb451d5

C:\Users\facun\Lab03-Docker-Facundo>docker rmi hello-world
Untagged: hello-world:latest
Deleted: sha256:452a468a4bf985040037cb6d5392410206e47db9bf5b7278d281f94d1c2d0931
```

Figura 24. Eliminación del contenedor y imágen

2.7 Creación de imagen personalizada

Se crea el directorio del proyecto.

```
C:\Users\facun\Lab03-Docker-Facundo>mkdir web-docker
```

Figura 25. Creación del directorio

Se define la configuración de la imagen personalizada.

index.html ● Dockerfile ●

Dockerfile > ...

```
1  FROM nginx:alpine
2  COPY index.html /usr/share/nginx/html/index.html
3  EXPOSE 80
4  CMD ["nginx", "-g", "daemon off;"]
```

Figura 25. Dockerfile

Se crea la página web a servir.

index.html ● Dockerfile ●

index.html > html

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Laboratorio 3 Docker</title>
</head>
<body>
    <h1>Laboratorio 3 - Docker</h1>
    <p>Facundo Rodríguez</p>
    <p>Contenedor creado con Dockerfile personalizado.</p>
</body>
</html>
```

Figura 26. index.html

Se construye la imagen utilizando docker build.

```
C:\Users\facun\Lab03-Docker-Facundo\web-docker>docker build -t facu-web .
[+] Building 11.3s (8/8) FINISHED                                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                                  0.3s
 => => transferring dockerfile: 151B                                                                                                  0.1s
 => [internal] load metadata for docker.io/library/nginx:alpine                                                                       3.4s
 => [auth] library/nginx:pull token for registry-1.docker.io                                                                          0.0s
 => [internal] load .dockerignore                                                                                                     0.1s
 => => transferring context: 2B                                                                                                       0.0s
 => [1/2] FROM docker.io/library/nginx:alpine@sha256:e7257f1ef28ba17cf7c248cb8ccf6f0c6e0228ab9c315c152f9c203cd34cf6d1                5.7s
 => => resolve docker.io/library/nginx:alpine@sha256:e7257f1ef28ba17cf7c248cb8ccf6f0c6e0228ab9c315c152f9c203cd34cf6d1                 0.1s
 => => sha256:3370263bc02adcf5c4f51831d2bf1d54dbf9a6a80b0bf32c5c9b9400630eaa08 20.25MB / 20.25MB                                     3.1s
 => => sha256:c2bd5ab177271dd59f19a46c214b1327f5c428cd075437ec0155ae71d0cdadc1 1.21kB / 1.21kB                                       0.7s
 => => sha256:4d9d41f3822d171ccc5f2cdfd75ad846ac4c7ed1cd36fb998fe2c0ce4501647b 1.40kB / 1.40kB                                       0.9s
 => => sha256:3f4ad4352d4f91018e2b4910b9db24c08e70192c3b75d0d6fff0120c838aa0bb 402B / 402B                                           1.0s
 => => sha256:cf1159c696ee2a72b85634360dbada071db61bceaad253db7fda65c45a58414c 953B / 953B                                           0.5s
 => => sha256:91d1c9c22f2c631288354fadb2decc448ce151d7a197c167b206588e09dcd50a 626B / 626B                                           0.4s
 => => sha256:8892f80f46a05d59a4cde3bcbb1dd26ed2441d4214870a4a7b318eaa476a0a54 1.87MB / 1.87MB                                       0.8s
 => => sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153 3.86MB / 3.86MB                                       1.0s
 => => extracting sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153                                             0.6s
 => => extracting sha256:8892f80f46a05d59a4cde3bcbb1dd26ed2441d4214870a4a7b318eaa476a0a54                                             0.5s
 => => extracting sha256:91d1c9c22f2c631288354fadb2decc448ce151d7a197c167b206588e09dcd50a                                             0.1s
 => => extracting sha256:cf1159c696ee2a72b85634360dbada071db61bceaad253db7fda65c45a58414c                                             0.1s
 => => extracting sha256:3f4ad4352d4f91018e2b4910b9db24c08e70192c3b75d0d6fff0120c838aa0bb                                             0.1s
 => => extracting sha256:c2bd5ab177271dd59f19a46c214b1327f5c428cd075437ec0155ae71d0cdadc1                                             0.1s
 => => extracting sha256:4d9d41f3822d171ccc5f2cdfd75ad846ac4c7ed1cd36fb998fe2c0ce4501647b                                             0.1s
 => => extracting sha256:3370263bc02adcf5c4f51831d2bf1d54dbf9a6a80b0bf32c5c9b9400630eaa08                                             1.3s
 => [internal] load build context                                                                                                     0.1s
 => => transferring context: 312B                                                                                                     0.0s
 => [2/2] COPY index.html /usr/share/nginx/html/index.html                                                                            0.2s
 => exporting to image                                                                                                                0.9s
 => => exporting layers                                                                                                               0.3s
 => => exporting manifest sha256:3bfe7f876227986e48c0dd4524638a574ab92bad88309278809a255a17ffa2a0                                     0.1s
 => => exporting config sha256:1a5335bbdf5e2b69f09cd81be0cbebb583b55a57dec0f7b3c121fce983eb2281                                       0.0s
 => => exporting attestation manifest sha256:3c2de709482f1140ba8ab096e28922e4f1b46c19295b34ab409ebc6cc2ac49b5                         0.1s
 => => exporting manifest list sha256:6c47fd2d3edba05af8b2d792d672952e9e91d58cd970a921489d598f0e72a358                               0.0s
 => => naming to docker.io/library/facu-web:latest                                                                                    0.0s
 => => unpacking to docker.io/library/facu-web:latest                                                                                 0.1s
```

Figura 27. Construcción de imagen

Se verifica la nueva imagen en el sistema.

```
C:\Users\facun\Lab03-Docker-Facundo\web-docker>docker images
                                                                                                    i Info →   U  In Use
IMAGE              ID             DISK USAGE   CONTENT SIZE   EXTRA
facu-web:latest    6c47fd2d3edb       92.6MB           26MB
nginx:latest       7150b3a39203        240MB         65.8MB    U
```

Figura 28. Imagen creada

Se ejecuta el contenedor personalizado.

```
C:\Users\facun\Lab03-Docker-Facundo\web-docker>docker run -d --name facu-web-contenedor -p 8090:80 facu-web
2a702316cd33f716f59803433462797e00c7a7a8c358ef6d79c85b97867804f7
```

Figura 29. Ejecución del contenedor

Se accede a la página personalizada desde el navegador.

Figura 30. Página web funcionando

Se visualizan los contenedores en ejecución.

```
C:\Users\facun\Lab03-Docker-Facundo\web-docker>docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED          STATUS          PORTS                                       NAMES
2a702316cd33   facu-web   "/docker-entrypoint.…"   49 seconds ago   Up 49 seconds   0.0.0.0:8090->80/tcp, [::]:8090->80/tcp     facu-web-contenedor
023630cc7770   nginx      "/docker-entrypoint.…"   24 minutes ago   Up 18 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp     mi-nginx
```

Figura 31. Contenedores activos

Se observan los registros del contenedor personalizado.

```
C:\Users\facun\Lab03-Docker-Facundo\web-docker>docker logs facu-web-contenedor
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/03/26 16:34:51 [notice] 1#1: using the "epoll" event method
2026/03/26 16:34:51 [notice] 1#1: nginx/1.29.7
2026/03/26 16:34:51 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
2026/03/26 16:34:51 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/03/26 16:34:51 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/03/26 16:34:51 [notice] 1#1: start worker processes
2026/03/26 16:34:51 [notice] 1#1: start worker process 30
2026/03/26 16:34:51 [notice] 1#1: start worker process 31
2026/03/26 16:34:51 [notice] 1#1: start worker process 32
2026/03/26 16:34:51 [notice] 1#1: start worker process 33
2026/03/26 16:34:51 [notice] 1#1: start worker process 34
2026/03/26 16:34:51 [notice] 1#1: start worker process 35
2026/03/26 16:34:51 [notice] 1#1: start worker process 36
2026/03/26 16:34:51 [notice] 1#1: start worker process 37
172.17.0.1 - - [26/Mar/2026:16:35:10 +0000] "GET / HTTP/1.1" 200 273 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) C
hrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0" "-"
2026/03/26 16:35:10 [error] 30#30: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 172.17.0.1, server: localhos
t, request: "GET /favicon.ico HTTP/1.1", host: "localhost:8090", referrer: "http://localhost:8090/"
172.17.0.1 - - [26/Mar/2026:16:35:10 +0000] "GET /favicon.ico HTTP/1.1" 404 555 "http://localhost:8090/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWeb
Kit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0" "-"
```

Figura 32. Logs del contenedor

2.8 Contenedor Ubuntu

Se ejecuta un contenedor interactivo basado en Ubuntu.

```
C:\Users\facun\Lab03-Docker-Facundo\web-docker>docker run -it --name ubuntu-lab ubuntu bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
817807f3c64e: Pull complete
e4a1e8de092c: Download complete
Digest: sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072feef0c
Status: Downloaded newer image for ubuntu:latest
root@d26aaf75878e:/#
```

Figura 33. Ejecución de Ubuntu

Se navega por el sistema de archivos.

```
C:\Users\facun\Lab03-Docker-Facundo\web-docker>docker run -it --name ubuntu-lab ubuntu bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
817807f3c64e: Pull complete
e4a1e8de092c: Download complete
Digest: sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072feef0c
Status: Downloaded newer image for ubuntu:latest
root@d26aaf75878e:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@d26aaf75878e:/# pwd
/
```

Figura 34. Exploración del sistema

Se actualiza la lista de paquetes.

```
root@d26aaf75878e:/# apt-get update
```

Figura 35. Actualización de paquetes

Se instala la herramienta curl.

```
root@d26aaf75878e:/# apt-get install -y curl
```

Figura 36. Instalación de curl

Se confirma la instalación de curl.

```
root@d26aaf75878e:/# curl --version
curl 8.5.0 (x86_64-pc-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 zlib/1.3 brotli/1.1.0 zstd/1.5.5 libidn2/2.3.7 libpsl/0.21.2 (+libidn2/2.3.7) libssh/0.10.6/op
enssl/zlib nghttp2/1.59.0 librtmp/2.3 OpenLDAP/2.6.10
Release-Date: 2023-12-06, security patched: 8.5.0-2ubuntu10.8
Protocols: dict file ftp ftps gopher gophers http https imap imaps ldap ldaps mqtt pop3 pop3s rtmp rtsp scp sftp smb smbs smtp smtps telnet tftp
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM PSL SPNEGO SSL threadsafe TLS-SRP UnixSockets zstd
root@d26aaf75878e:/#
```

Figura 37. Verificación de curl

2.9 Contenedor Alpine

Se ejecuta un contenedor basado en Alpine Linux.

```
C:\Users\facun\Lab03-Docker-Facundo\web-docker>docker run -it --name alpine-lab alpine sh
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
caa817ad3aea: Download complete
9e595aac14e0: Download complete
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
/ # ls
bin    dev    etc    home   lib    media  mnt    opt    proc   root   run    sbin   srv    sys    tmp    usr    var
/ # pwd
/
/ #
```

Figura 38. Ejecución de Alpine

Se actualizan los paquetes con apk.

```
/ # apk update
v3.23.3-381-g347abb1c44d [https://dl-cdn.alpinelinux.org/alpine/v3.23/main]
v3.23.3-381-g347abb1c44d [https://dl-cdn.alpinelinux.org/alpine/v3.23/community]
OK: 27583 distinct packages available
/ #
```

Figura 39. Actualización con apk

Se instala curl utilizando apk.

```
/ # apk add curl
( 1/10) Installing brotli-libs (1.2.0-r0)
( 2/10) Installing c-ares (1.34.6-r0)
( 3/10) Installing libunistring (1.4.1-r0)
( 4/10) Installing libidn2 (2.3.8-r0)
( 5/10) Installing nghttp2-libs (1.68.0-r0)
( 6/10) Installing nghttp3 (1.13.1-r0)
( 7/10) Installing libpsl (0.21.5-r3)
( 8/10) Installing zstd-libs (1.5.7-r2)
( 9/10) Installing libcurl (8.17.0-r1)
(10/10) Installing curl (8.17.0-r1)
Executing busybox-1.37.0-r30.trigger
OK: 13.2 MiB in 26 packages
```

Figura 40. Instalación de curl en Alpine

Se verifica la instalación de curl en Alpine.

```
curl 8.17.0 (x86_64-alpine-linux-musl) libcurl/8.17.0 OpenSSL/3.5.5 zlib/1.3.1 brotli/1.2.0 zstd/1.5.7 c-ares/1.34.6 libidn2/2.3.8 libpsl/0.21.5 nghttp2/1.6
8.0 nghttp3/1.13.1
Release-Date: 2025-11-05
Protocols: dict file ftp ftps gopher gophers http https imap imaps ipfs ipns mqtt pop3 pop3s rtsp smb smbs smtp smtps telnet tftp ws wss
Features: alt-svc AsynchDNS brotli HSTS HTTP2 HTTP3 HTTPS-proxy IDN IPv6 Largefile libz NTLM PSL SSL threadsafe TLS-SRP UnixSockets zstd
/ #
```

Figura 41. Verificación de curl

Se visualizan todas las imágenes descargadas.

```
C:\Users\facun\Lab03-Docker-Facundo\web-docker>docker images
                                                                                    Info →   U  In Use
IMAGE              ID            DISK USAGE   CONTENT SIZE   EXTRA
alpine:latest      25109184c71b       13.1MB         3.95MB    U
facu-web:latest    6c47fd2d3edb       92.6MB           26MB    U
nginx:latest       7150b3a39203        240MB         65.8MB    U
ubuntu:latest      186072bba1b2        119MB         31.7MB    U
```

Figura 42. Listado final de imágenes

2.10 Publicación en GitHub

Se crea el repositorio del laboratorio.

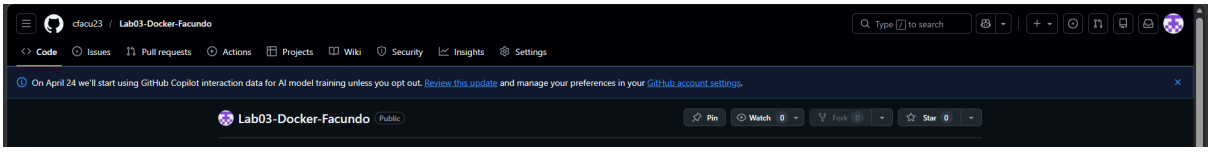

Figura 43. Repositorio en GitHub

Se suben Dockerfile, index.html y README.

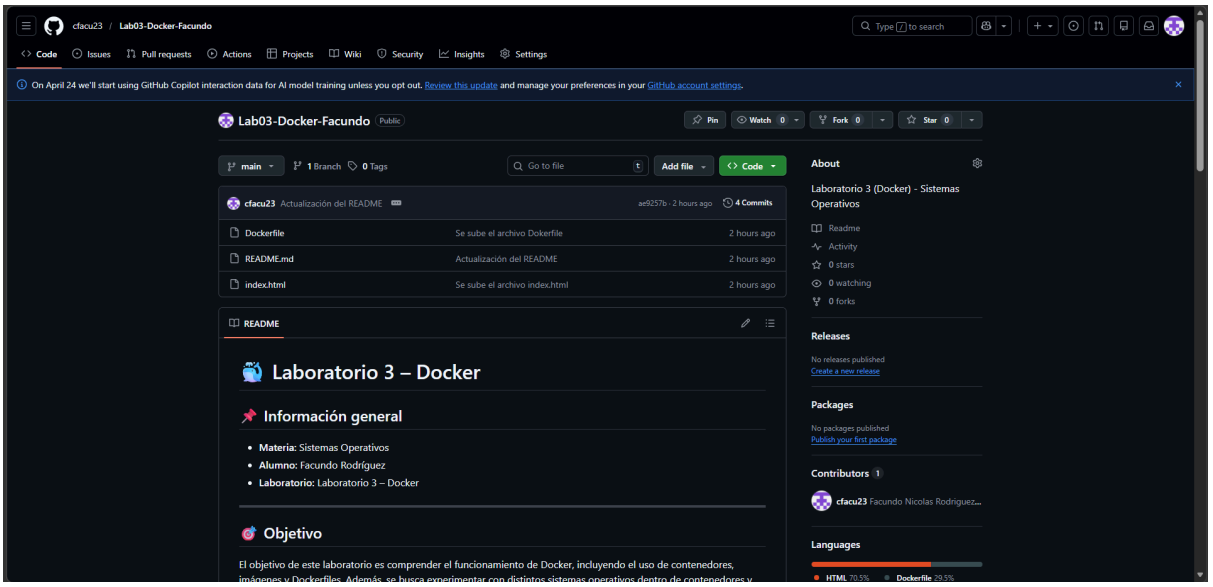

Figura 44. Archivos subidos

3. Introducción a Docker

Docker es una herramienta que permite ejecutar aplicaciones dentro de contenedores, facilitando su desarrollo, prueba y despliegue en distintos entornos. Su principal ventaja es que permite trabajar con entornos aislados, asegurando que una aplicación funcione de la misma manera independientemente del sistema donde se ejecute.

Entre sus ventajas se destacan la portabilidad, la eficiencia en el uso de recursos y la rapidez en la ejecución, ya que los contenedores son mucho más livianos que las máquinas virtuales tradicionales.

Dentro de Docker existen tres conceptos fundamentales:

- **Imagen**: es una plantilla que contiene todo lo necesario para ejecutar una aplicación, incluyendo sistema operativo, dependencias y configuraciones.
- **Contenedor**: es una instancia en ejecución de una imagen. Funciona como un entorno aislado donde se ejecuta la aplicación.
- **Dockerfile**: es un archivo de instrucciones que permite crear imágenes personalizadas de forma automatizada.

Estos tres elementos están directamente relacionados, ya que a partir de un Dockerfile se crea una imagen, y a partir de esa imagen se ejecutan uno o más contenedores.

4. Comandos fundamentales de Docker

Para trabajar con Docker es necesario conocer los comandos básicos que permiten gestionar contenedores e imágenes:

- **docker ps**: permite listar los contenedores que se encuentran en ejecución.
- **docker ps -a**: muestra todos los contenedores, incluyendo los que están detenidos.
- **docker stop <nombre_o_ID>**: detiene un contenedor en ejecución.
- **docker start <nombre_o_ID>**: inicia un contenedor que se encuentra detenido.
- **docker rm <nombre_o_ID>**: elimina un contenedor del sistema.
- **docker images**: lista todas las imágenes disponibles localmente.
- **docker rmi <nombre_o_ID>**: elimina una imagen de Docker.
- **docker logs <nombre_o_ID>**: muestra los registros de ejecución de un contenedor.
- **docker exec -it <nombre_o_ID> <comando>**: permite ejecutar comandos dentro de un contenedor en ejecución de forma interactiva.

Estos comandos permiten interactuar con los contenedores, administrarlos y verificar su funcionamiento.

5. Dockerfiles: propósito y estructura

El Dockerfile es un archivo de texto que contiene una serie de instrucciones que Docker utiliza para construir imágenes personalizadas. Su propósito principal es automatizar la creación de imágenes, permitiendo replicar entornos de forma consistente y sencilla.

Un Dockerfile está compuesto por una secuencia de instrucciones que se ejecutan en orden, cada una representando una capa en la imagen final. Su sintaxis se basa en comandos específicos que indican qué acciones realizar durante la construcción de la imagen.

6. Instrucciones más comunes de un Dockerfile

A continuación, se detallan las instrucciones más utilizadas dentro de un Dockerfile:

- **FROM**: especifica la imagen base sobre la cual se construirá la nueva imagen.
- **RUN**: ejecuta comandos dentro del contenedor durante el proceso de construcción.
- **CMD**: define el comando por defecto que se ejecutará cuando se inicie el contenedor.

- **ENTRYPOINT**: similar a CMD, pero establece el punto de entrada principal del contenedor.
- **COPY**: copia archivos o directorios desde el sistema local al contenedor.
- **ADD**: similar a COPY, pero también permite descomprimir archivos o descargar desde URLs.
- **WORKDIR**: establece el directorio de trabajo dentro del contenedor.
- **EXPOSE**: indica los puertos que el contenedor utilizará en tiempo de ejecución.
- **ENV**: define variables de entorno dentro del contenedor.
- **ARG**: define variables que pueden pasarse durante la construcción de la imagen.
- **VOLUME**: crea un punto de montaje para persistencia de datos.
- **USER**: establece el usuario que ejecutará los comandos dentro del contenedor.
- **LABEL**: agrega metadatos a la imagen.
- **STOPSIGNAL**: define la señal que se enviará al contenedor para detenerlo.
- **HEALTHCHECK**: permite verificar el estado de funcionamiento del contenedor.
- **SHELL**: permite modificar el shell utilizado para ejecutar comandos.

Estas instrucciones permiten definir completamente el comportamiento y configuración de una imagen Docker.

7. Aplicación práctica de los comandos

A lo largo del laboratorio se utilizaron los comandos mencionados para interactuar con contenedores, ejecutarlos, detenerlos, eliminarlos y verificar su funcionamiento. Esto permitió comprender de forma práctica cómo Docker gestiona los recursos y cómo se puede trabajar con múltiples contenedores de forma simultánea.

8. Conclusión

En este laboratorio se logró comprender el funcionamiento de Docker y su utilidad para ejecutar aplicaciones en entornos aislados mediante contenedores. Se trabajó con la instalación de Docker, la ejecución de contenedores, la gestión de imágenes y la creación de una imagen personalizada mediante Dockerfile.

Además, se experimentó con distintos sistemas operativos como Ubuntu y Alpine, observando diferencias en su funcionamiento y gestión de paquetes. Esto permitió reforzar el concepto de portabilidad y eficiencia que Docker ofrece.

En conclusión, Docker se presenta como una herramienta fundamental en el desarrollo moderno de software, facilitando la implementación y despliegue de aplicaciones.

9. Anexos

Repositorio: https://github.com/cfacu23/Lab03-Docker-Facundo